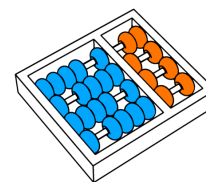




Relatório de desenvolvimento das atividades
Instituto de Computação - Universidade Estadual de Campinas
Allan M. de Souza, Rafael O. Jarczewski



Nome: José Eduardo Santos Rabelo
RA: 260551

Descrição de como foi realizado o ataque com prints. Deve ser capaz de replicar o ataque seguindo a sequência de passos utilizada no ambiente disponibilizado.

O experimento foi realizado em um ambiente isolado com três contêineres Docker interligados por uma rede virtual 10.0.2.0/24. Esse ambiente reproduz, de forma acadêmica e controlada, um cenário clássico de confiança entre hosts, no qual um servidor alvo aceita conexões originadas de um host previamente confiável. A topologia utilizada foi composta por três papéis bem definidos: a máquina atacante, o servidor confiável e o xTerminal, que representa o alvo final da atividade.

No início da execução, foi realizada a inicialização completa dos contêineres com docker compose, garantindo que os três serviços estivessem ativos e conectados à mesma rede virtual. Após essa etapa, foi feita a verificação de conectividade entre os hosts para confirmar que a comunicação interna do laboratório estava funcionando corretamente. Essa validação foi importante para assegurar que qualquer comportamento observado posteriormente fosse decorrente da lógica implementada no experimento, e não de falhas de rede ou de configuração do ambiente.

Em seguida, foi confirmada a relação de confiança configurada no xTerminal. Essa verificação foi feita por meio da leitura do arquivo `.rhosts` presente no contêiner alvo, no qual estava registrada a autorização do host 10.0.2.30 para acesso sem autenticação interativa. Essa evidência foi essencial para demonstrar que o laboratório realmente reproduzia a premissa central do cenário estudado: a confiança implícita baseada no endereço IP de origem.

Depois da validação estrutural do ambiente, foi executado o código desenvolvido no diretório `mitnick/`. A implementação foi organizada em etapas, com um script principal responsável por coordenar a atividade e módulos auxiliares encarregados de tarefas específicas observadas no cenário. No contexto do relatório, o ponto principal não é detalhar comandos de exploração, mas registrar que a sequência do experimento foi conduzida de forma ordenada, dentro do ambiente disponibilizado, e com observação contínua das respostas de rede obtidas no próprio laboratório.

O primeiro resultado relevante observado durante a execução foi a confirmação de que a máquina atacante conseguia estabelecer o início de uma comunicação TCP com os serviços presentes no cenário. Para isso, foi utilizado um script em Python com a biblioteca Scapy, que permitiu montar e enviar pacotes manualmente e observar as respostas retornadas pelo host de destino. Esse teste foi importante porque demonstrou, na prática, o funcionamento do handshake TCP no ambiente montado e serviu como base para a análise do comportamento dos pacotes trocados entre os nós.

Ao longo do procedimento, a atividade foi documentada por meio de saídas de terminal e prints do ambiente. Essas evidências visuais foram consideradas parte fundamental do experimento, pois permitem comprovar o estado do ambiente, a conectividade entre os hosts, a configuração da relação de confiança e o comportamento das respostas TCP observadas. Por esse motivo, o relatório foi estruturado para incorporar tanto a explicação textual quanto os registros visuais correspondentes a cada fase importante.

Em síntese, a atividade foi conduzida em quatro momentos principais: preparação do ambiente, validação de rede, confirmação da relação de confiança e observação do comportamento TCP por meio da execução dos scripts desenvolvidos. Dessa forma, foi possível documentar o cenário de maneira coerente com o objetivo acadêmico do laboratório, preservando o foco na análise técnica do ambiente controlado e na interpretação das respostas observadas.

```
● eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it attacker ping -c 2 10.0.2.20
docker exec -it attacker ping -c 2 10.0.2.30

PING 10.0.2.20 (10.0.2.20) 56(84) bytes of data.
64 bytes from 10.0.2.20: icmp_seq=1 ttl=64 time=3.58 ms
64 bytes from 10.0.2.20: icmp_seq=2 ttl=64 time=0.116 ms

--- 10.0.2.20 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1008ms
rtt min/avg/max/mdev = 0.116/1.849/3.583/1.733 ms
PING 10.0.2.30 (10.0.2.30) 56(84) bytes of data.
64 bytes from 10.0.2.30: icmp_seq=1 ttl=64 time=0.191 ms
64 bytes from 10.0.2.30: icmp_seq=2 ttl=64 time=0.084 ms

--- 10.0.2.30 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1046ms
rtt min/avg/max/mdev = 0.084/0.137/0.191/0.053 ms
```

Figura 1 – Teste de conectividade do contêiner attacker para target (10.0.2.20) e server (10.0.2.30), com 0% de perda de pacotes, validando a comunicação na rede do laboratório.

```
● eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it target cat /root/.rhosts
10.0.2.30 root
```

Figura 2 – Conteúdo do arquivo /root/.rhosts no target, evidenciando a relação de confiança pré-configurada para o host 10.0.2.30 (usuário root).

```

● eduardosrbl@Eduardos-MacBook-Air lab03 % docker cp ./mitnick/. attacker:/app
docker exec -it attacker python3 /app/attack.py

Successfully copied 27.6kB to attacker:/app
[*] Iniciando SYN Flood em 10.0.2.30:514 por 10s...
[*] 9.5s | Pacotes enviados: 963
[+] Flood concluído: 1021 pacotes em 10.1s (101 pkt/s)
[+] Servidor Confiável (10.0.2.30) deve estar sobrecarregado.

```

Figura 3 – Execução do script attack.py no attacker, com envio contínuo de pacotes SYN para 10.0.2.30:514 e resumo final do flood (quantidade de pacotes e taxa média).

```

● eduardosrbl@Eduardos-MacBook-Air lab03 % docker cp ./mitnick/. attacker:/app
docker exec -it attacker python3 /app/isn_probe.py

Successfully copied 27.6kB to attacker:/app
[*] Tentativa 1/4 (sport=514)
[+] ARP poison: 10.0.2.30 → 5e:81:2f:9a:3d:fa (enviado a 10.0.2.20)
[-] Nenhum SYN-ACK capturado nesta tentativa.
[*] Tentativa 2/4 (sport=773)
[+] ARP poison: 10.0.2.30 → 5e:81:2f:9a:3d:fa (enviado a 10.0.2.20)
[+] SYN-ACK capturado de 10.0.2.20:514
[+] ISN (seq do xTerminal): 1608039626
[+] ACK esperado para completar handshake: 1608039627

```

Figura 4 – Execução do script isn_probe.py no attacker, mostrando tentativa de captura, sucesso no SYN-ACK e extração do ISN (seq) com cálculo do ACK esperado.

```

● eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it target tcpdump -n -i any tcp port 514

tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
17:34:31.385190 eth0 In IP 10.0.2.10.48744 > 10.0.2.20.514: Flags [S], seq 1436550667, win 64240, options [mss 1460,sackOK,TS val 341605824 ecr 0,nop,wscale 7], length 0
17:34:31.385292 eth0 Out IP 10.0.2.20.514 > 10.0.2.10.48744: Flags [S.], seq 1158122665, ack 1436550668, win 65160, options [mss 1460,sackOK,TS val 3061601155 ecr 341605824,nop,wscale 7], length 0
17:34:31.385249 eth0 In IP 10.0.2.10.48744 > 10.0.2.20.514: Flags [F.], ack 1, win 502, options [nop,nop,TS val 341605825 ecr 3061601155], length 0
17:34:31.385329 eth0 In IP 10.0.2.10.48744 > 10.0.2.20.514: Flags [F.], seq 1, ack 1, win 502, options [nop,nop,TS val 341605825 ecr 3061601155], length 0
17:34:31.385296 eth0 Out IP 10.0.2.20.514 > 10.0.2.10.48744: Flags [F.], ack 2, win 510, options [nop,nop,TS val 3061601156 ecr 341605825], length 0
17:34:31.387470 eth0 Out IP 10.0.2.20.514 > 10.0.2.10.48744: Flags [F.], seq 1, ack 2, win 510, options [nop,nop,TS val 3061601157 ecr 341605825], length 0
17:34:31.387488 eth0 In IP 10.0.2.10.48744 > 10.0.2.20.514: Flags [F.], ack 2, win 502, options [nop,nop,TS val 341605827 ecr 3061601157], length 0
17:34:31.942314 eth0 In IP 10.0.2.30.918 > 10.0.2.20.514: Flags [S], seq 1000, win 8192, length 0
17:34:31.942336 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:31.997654 eth0 In IP 10.0.2.30.918 > 10.0.2.20.514: Flags [S], seq 1000, win 8192, length 0
17:34:31.997690 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:33.006377 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:35.050489 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:39.082012 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:47.469000 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:35:03.850751 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0

```

Figura 5 – Captura de tráfego no target com tcpdump na porta 514, mostrando pacotes SYN de entrada e respostas SYN-ACK de saída, confirmando o comportamento TCP observado no experimento.

Detalhe como foi o processo para obtenção do número de sequência (ISN).

O Número de Sequência Inicial, ou ISN (*Initial Sequence Number*), é um campo do protocolo TCP utilizado para iniciar o controle de sequência dos dados durante o estabelecimento de uma conexão. No contexto do experimento, a identificação desse valor foi importante porque permitiu analisar como o host de destino respondia ao início de uma tentativa de conexão e quais informações eram devolvidas no cabeçalho TCP.

Para compreender esse processo, é necessário lembrar o funcionamento básico do three-way handshake do TCP. Quando uma conexão é iniciada, o primeiro host envia um segmento com a flag SYN. O host de destino, ao aceitar a tentativa de conexão, responde com um segmento SYN-ACK. É exatamente nessa resposta que aparece o número de sequência inicial gerado pelo destino. Por fim, a conexão é completada quando o emissor devolve um ACK.

No laboratório, a observação do ISN foi feita a partir da resposta retornada pelo host alvo em uma interação controlada dentro da rede virtual. O processo adotado consistiu em induzir o envio de uma resposta TCP pelo xTerminal e, em seguida, capturar essa resposta para inspeção dos campos do cabeçalho. Assim, em vez de tratar o ISN de forma abstrata, o experimento permitiu visualizar concretamente onde esse valor aparece e como ele é representado no pacote de retorno.

Na implementação desenvolvida, a biblioteca Scapy foi utilizada para criar e inspecionar pacotes. Após o envio do segmento inicial, a resposta de retorno foi capturada e analisada no próprio ambiente. O campo mais importante dessa análise foi o seq do pacote SYN-ACK, pois ele representa justamente o número de sequência inicial escolhido pelo host que respondeu. Além dele, também foram observados os campos ack e flags, que ajudam a confirmar que o segmento capturado correspondia ao segundo passo do handshake TCP.

Dessa forma, o processo de obtenção do ISN no laboratório pode ser descrito, de forma objetiva, como uma sequência de observação do comportamento do protocolo: primeiro foi gerada uma tentativa de abertura de conexão; depois foi coletada a resposta correspondente; por fim, essa resposta foi inspecionada para identificar o valor presente no campo seq. O resultado dessa análise mostrou, na prática, como o número de sequência inicial pode ser observado no tráfego TCP dentro de uma rede local simulada e controlada.

É importante destacar que, neste relatório, o foco está na explicação técnica da observação do campo seq e na interpretação do comportamento do protocolo dentro do ambiente acadêmico. Assim, o objetivo desta seção é demonstrar entendimento sobre onde o ISN aparece, por que ele é relevante e como ele foi identificado a partir da resposta TCP observada durante o experimento.